



Smart Wardrobe Planner

Digitaler Kleiderschrank & Outfit-Bewertung

Doaa Al-Shoumi & Sameer Rana

Grundlagen des World Wide Web · SS 2026

Betreuer: PD Dr. Alexander Hinneburg

Das Problem



Idee

Jeden Morgen die Frage: Was ziehe ich an? Ein digitaler Kleiderschrank, der Outfits bewertet und die Entscheidung erleichtert.



Herausforderung

Kleidung ist über viele Kategorien verteilt · Kein Überblick über Kombinationen · Zeitaufwändiges manuelles Zusammenstellen

50+

Kleidungsstücke mit echten Bildern

4

Bewertungsdimensionen auf einen Blick



Live-Radar bei jeder Auswahl

App-Architektur



Drei Seiten

Home, Closet und About - alle über die Browser-URL erreichbar mit Vor-/Zurück-Navigation



Daten aus JSON

50+ Kleidungsstücke werden aus einer JSON-Datei geladen und typsicher in die App-Struktur übersetzt



Interaktives SVG

Das Radar-Diagramm ist ein echtes SVG und zeichnet sich bei jeder Änderung der Auswahl automatisch neu



Drei Schichten

Zustand (Model), Logik (Update) und Darstellung (View) sind strikt getrennt - wartbar und erweiterbar

⚙ Elm · Bulma CSS (responsiv) · GitLab Pages

So funktioniert das Radar-Diagramm

Ein Radar-Diagramm zeigt auf einen Blick, wie gut ein Outfit in verschiedenen Kategorien abschneidet - je größer die Fläche, desto ausgewogener das Outfit.

Definition der Scores

```
type alias Scores =  
  { style : Int, comfort : Int  
    , formal : Int, durable : Int }
```

Berechnung des Mittelwerts

```
combineScores items =  
  Scores (average .style items)  
        (average .comfort items)  
        (average .formal items)  
        (average .durable items)
```

Umwandlung in SVG

```
dataPolygon cx cy r axes =  
  polygon [ points pts  
            , fill "#9FBF47"  
            , fillOpacity "0.35" ] []
```

Live-Update

Jede Änderung der Auswahl löst
eine Neuberechnung aus:
Item wählen -> Model ändert sich
-> View rendert neu -> Polygon
aktualisiert sich automatisch

Ergebnis & Ausblick



Elm als Basis

Typensicherheit verhindert Laufzeitfehler · Saubere Trennung von Zustand, Logik und Darstellung



Erreichte Ziele

Voll funktionsfähiger Prototyp mit 3 Routen · HTTP-Ladung der JSON-Daten · Interaktives SVG-Radar in Echtzeit



Optimierung

CSS-Styling aufwändiger · Früher mit Datenstruktur beginnen · Elm-Paketdokumentation gewöhnungsbedürftig



Ausblick

Wetter-API für Vorschläge · Outfits als JSON speichern · Filter nach Anlass (Uni, Party, Business)